"Authenticate first, then authorize -- every request, every time."

MODULE 28

Spring Security Fundamentals

Learn core security concepts and implement robust authentication and authorization using Spring Security.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

WHY APPLICATION SECURITY MATTERS

Modern applications store and process sensitive data, handle critical business transactions, and are exposed to the internet.

Security is no longer optional -- it is essential.

THE IMPACT OF POOR SECURITY

Impact	What Happens
Data Breaches	Exposure of sensitive customer, financial, and business data.
Financial Loss	Direct losses, fraud, legal fees, and regulatory penalties.
Reputation Damage	Loss of customer trust and long-term brand impact.
Compliance Violations	Failure to meet security standards and industry regulations.
Service Disruption	Attacks can cause downtime and disrupt business operations.

KEY REASONS SECURITY MATTERS

- **Protects Sensitive Data —** safeguards personal, financial, and business information from unauthorized access.
- **Ensures Business Continuity —** prevents attacks that can disrupt services and impact availability.
- **Builds Customer Trust —** secure applications show customers their data is safe.
- **Meets Regulatory Requirements —** helps comply with standards like GDPR, PCI-DSS, HIPAA.
- **Supports Secure Innovation —** security by design enables confident delivery of new features.

KEY TAKEAWAY Strong application security protects your users, your data, and your business -- today and in the future.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to build secure, role-based Spring Boot applications with confidence.

- **Explain the Fundamentals —** of application security and the difference between authentication and authorization.
- **Identify Common Threats —** security threats and web application vulnerabilities and how they impact applications.
- **Describe the Architecture —** Spring Security architecture and how the security filter chain processes requests.
- **Implement Authentication —** using Spring Security and configure secure login and password encoding.
- **Implement Authorization —** using role-based access control (RBAC) and secure application endpoints.
- **Understand JWT —** JSON Web Token structure, and implement JWT-based authentication and token validation.
- **Apply Best Practices —** and avoid common security mistakes in enterprise applications.

KEY TAKEAWAY Building confidence in secure Spring applications, protecting data and endpoints, and adhering to enterprise security standards.

AUTHENTICATION vs AUTHORIZATION

Two different but connected concepts that work together to secure your application.

	Authentication	Authorization
Question	"Who are you?"	"What are you allowed to do?"
Goal	Verify the identity of the user.	Determine what the user is allowed to access or do.
Examples	Login with username/password, MFA, social login, API key/token.	RBAC, permissions (READ/WRITE/DELETE), securing endpoints, feature access.
Outcome	User is authenticated -- the system knows who you are.	User is authorized -- the system knows what you can do.

HOW THEY WORK TOGETHER

1. **User Attempts Access**
2. **Authentication**
3. **Authorization**
4. **Access Granted / Denied**

KEY TAKEAWAY Authentication verifies who the user is. Authorization determines what the user can do. Both are essential for building secure and reliable applications.

TRY IT YOURSELF -- EXERCISE 1: AUTHENTICATION VS AUTHORIZATION

Reinforces: separating 401 (not authenticated) from 403 (authenticated but forbidden) with real Northstar agent/admin examples.

STEP	WHAT TO DO
Goal	Explain 401 vs 403 with Northstar agent/admin examples -- in notes/authn-authz.md.
Define	Write authentication and authorization in one sentence each -- who you are vs. what you can do.
Reference row	Fill a 401/403/200 example matching: 401 = not authenticated, 403 = authenticated but forbidden, 200 = allowed.
Lab users	Record agent1 (AGENT) and admin1 (ADMIN) -- the two roles used throughout Lab 28.
Correlation != auth	lab-request-001 is operational metadata for tracing a request -- never treat it as a credential.
Reference	exercises/exercise-01-authn-vs-authz.md

Reference Matrix (fill this in)

```
Status  Meaning                 CRM Example
401     Not authenticated       No/invalid Bearer token
403     Authenticated, forbidden agent1 hits /api/admin/**
200     Allowed                 agent1 GET CUS-1001
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 1 before continuing -- define authn vs. authz, fill the 401/403/200 example, and record agent1/admin1 -- see exercises/exercise-01-authn-vs-authz.md.

SECURITY THREAT LANDSCAPE

Modern applications face a constantly evolving set of threats. Understanding the landscape is the first step to building secure, resilient applications.

COMMON THREAT CATEGORIES

1. Malicious Actors

Hackers, insiders, and cybercriminals targeting apps for financial gain or disruption.

2. Vulnerabilities

Flaws in code, frameworks, or dependencies exploitable for unauthorized access.

3. Social Eng.

Tricking users into revealing sensitive information (phishing, pretexting).

4. Data Exposure

Unauthorized access, leakage, or theft of data due to weak controls.

5. Service Disrupt.

Attacks aimed at making apps unavailable (denial-of-service).

6. Third-Party Risk

Risks from third-party APIs, libraries, and integrations with their own weaknesses.

POTENTIAL BUSINESS IMPACT

Financial Loss | **Reputation Damage** | **Regulatory Penalties** | **Operational Disruption** | **Competitive Disadvantage**

Security is not just an IT concern -- it's a business imperative. The threat landscape is dynamic; proactive awareness, secure coding, and defense-in-depth are essential.

KEY TAKEAWAY The threat landscape is dynamic and ever-changing. Proactive security awareness, secure coding, and defense-in-depth are essential.

COMMON WEB SECURITY RISKS (1 OF 2)

Web applications are frequent targets for attackers. Understanding common risks helps us build secure and resilient applications.

Risk	Description	Example
1. Injection Attacks	Attackers inject malicious code via user input to manipulate queries or execute unauthorized commands.	SQL Injection, NoSQL Injection, Command Injection
2. Cross-Site Scripting (XSS)	Malicious scripts execute in the user's browser to steal cookies, session tokens, or deface pages.	Stored XSS, Reflected XSS, DOM-based XSS
3. Broken Authentication	Weak login mechanisms allow attackers to gain unauthorized access to accounts or data.	Weak passwords, session hijacking, credential stuffing
4. Broken Access Control	Improper restrictions allow users to access resources or perform actions outside their permissions.	IDOR (Insecure Direct Object Reference), privilege escalation

WHY THIS MATTERS FOR SPRING SECURITY

Spring Security directly addresses risks 3 and 4 (authentication and access control) -- risks 1, 2, and beyond require secure coding practices on top of the framework.

KEY TAKEAWAY No application is 100% secure, but awareness and best practices significantly reduce risk.

COMMON WEB SECURITY RISKS (2 OF 2)

Risks 5-7, and the best practices that address all seven.

Risk	Description	Example
5. Security Misconfiguration	Unsafe default settings, open services, or verbose error messages expose sensitive information.	Default credentials, unnecessary services, verbose errors
6. Sensitive Data Exposure	Failure to protect sensitive data can lead to leakage of personal, financial, or business information.	Unencrypted data, weak encryption, data in logs
7. Cross-Site Request Forgery (CSRF)	Attackers trick authenticated users into performing unwanted actions on a web application.	Transfer funds, change email, update settings

BEST PRACTICES ACROSS ALL SEVEN RISKS

- **Validate & Sanitize Input**
- **Strong Authn & Session Mgmt**
- **Least Privilege Access**
- **Keep Dependencies Updated**
- **Monitor, Log, Respond**

KEY TAKEAWAY No application is 100% secure, but awareness and best practices significantly reduce risk.

SPRING SECURITY OVERVIEW (1 OF 2)

Spring Security is the de-facto security framework for Spring-based applications, providing authentication, authorization, and protection against common attacks.

A powerful and highly customizable authentication and access-control framework, focused on providing both authentication and authorization to Java applications.

WHY SPRING SECURITY?

- **Industry Standard —** widely adopted in enterprise applications.
- **Deep Spring Integration —** works seamlessly with Spring Boot, Spring MVC, and Spring Cloud.
- **Comprehensive Security —** covers authentication, authorization, CSRF protection, session management, and more.
- **Flexible & Powerful —** easy to configure, yet highly customizable.

KEY FEATURES

Authentication

Supports multiple authentication mechanisms.

Authorization

Fine-grained access control using roles and permissions.

Protection

Built-in protection against common web threats.

Integration

Seamless integration with the Spring ecosystem.

Extensibility

Highly configurable and extensible architecture.

KEY TAKEAWAY Spring Security provides a robust, flexible, and extensible foundation for securing modern Java applications.

SPRING SECURITY OVERVIEW (2 OF 2)

How a request flows through Spring Security's modules on its way to your protected resource.

SPRING SECURITY MODULES (REQUEST FLOW)

1. **Client** — HTTP Request
2. **Security Filter Chain** — Ordered filters
3. **Authentication Manager** — Verifies identity
4. **Authorization Manager** — Checks permissions
5. **Security Context** — Protected resource

AUTHENTICATION PROVIDERS

- UserDetailsService (in-memory, JDBC, JPA)
- DAO authentication provider
- LDAP
- OAuth2 / JWT

ACCESS DECISION MANAGERS

- Role-Based Access
- Voters
- Policies
- Permissions

KEY TAKEAWAY Spring Security provides a robust, flexible, and extensible foundation for securing modern Java applications.

SECURITY FILTER CHAIN (1 OF 2)

Spring Security uses a filter chain to intercept and process every incoming HTTP request.

A Security Filter Chain is an ordered list of filters that securitize requests and responses before they reach your application. Each filter has a specific responsibility and passes the request to the next filter.

DEFAULT FILTER CHAIN FLOW

1. **SecurityContext Filter**
2. **UsernamePassword Filter**
3. **Authorization Filter**

1. **Exception Translation**
2. **Security Interceptor**

KEY POINTS

- Every request goes through the filter chain, in a predefined order.
- Filters perform tasks like authentication, authorization, CSRF protection, session management.
- If a filter handles the request, the chain stops there; if not, the request continues to the next filter.

KEY TAKEAWAY The Security Filter Chain is the backbone of Spring Security, ensuring every request is authenticated, authorized, and protected before reaching your application.

SECURITY FILTER CHAIN (2 OF 2)

Common filters in the chain, their purpose, and how the chain processes a request end to end.

COMMON FILTERS IN THE CHAIN

Filter	Purpose
SecurityContextPersistenceFilter	Loads and stores the security context.
UsernamePasswordAuthenticationFilter	Authenticates username and password logins.
BasicAuthenticationFilter	Handles HTTP Basic Authentication.
CsrfFilter	Protects against Cross-Site Request Forgery.
ExceptionTranslationFilter	Converts security exceptions to HTTP responses.
FilterSecurityInterceptor	Enforces authorization rules for protected resources.

HOW IT WORKS

- 1. Request enters the application.
- 2. It passes through each filter in the chain, in order.
- 3. Each filter performs its task (authenticate, authorize, validate).
- 4. If a filter rejects the request, an error/redirect response is returned.
- 5. If all filters approve, the request reaches the target resource (controller/service).

KEY TAKEAWAY The Security Filter Chain is the backbone of Spring Security, ensuring every request is authenticated, authorized, and protected before reaching your application.

TRY IT YOURSELF -- EXERCISE 2: SECURITYFILTERCHAIN SKETCH

Reinforces: sketching Lab 28's four security components and route rules -- design only, no implementation yet.

STEP	WHAT TO DO
Goal	Sketch Lab 28's security components in notes/filter-chain.md without implementing them.
Components	List all four: SecurityFilterChain, JwtService, JwtAuthenticationFilter, CrmUserDetailsService.
Session policy	Write: session creation policy is STATELESS for JWT APIs -- no HttpSession is used.
Route rules	/api/auth/login permitAll(); /api/customers/** AGENT or ADMIN; /api/admin/** ADMIN only.
CSRF note	For stateless Bearer APIs, CSRF is typically disabled -- confirm the reasoning in the lab guide.
Reference	exercises/exercise-02-filter-chain-sketch.md

Four Components (fill notes/filter-chain.md)

```
SecurityFilterChain      -- authorizes HTTP requests
JwtService               -- issues and parses tokens
JwtAuthenticationFilter  -- reads the Bearer header
CrmUserDetailsService    -- loads lab users/roles
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 2 before continuing -- list the four components, note STATELESS, and write the three route rules -- see exercises/exercise-02-filter-chain-sketch.md.

REQUEST PROCESSING FLOW (1 OF 2)

Spring Security intercepts every HTTP request through the Security Filter Chain and ensures only authenticated and authorized requests reach your application.

Spring Security sits in front of your application. It authenticates the user, authorizes the request, and protects your resources.

1. **Filter Chain Entry**
   Request enters the chain
2. **Authentication**
   Validate identity
3. **Authorization**
   Check permission

1. **Other Filters**
   CSRF, session, CORS
2. **Controller / Resource**
   If all checks pass
3. **Response**
   Response sent back

KEY TAKEAWAY The request processing flow ensures that every request is authenticated, authorized, and validated before reaching your application -- keeping your data and users secure.

REQUEST PROCESSING FLOW (2 OF 2)

What happens when authentication or authorization fails, and the filters most commonly involved.

WHEN THINGS GO WRONG

Failure	Result
Authentication Failed	User is not authenticated. Return 401 Unauthorized (or redirect to login for browser apps).
Access Denied	User is authenticated but not authorized. Return 403 Forbidden.

EXAMPLE FILTERS IN THE CHAIN

- **SecurityContextPersistenceFilter —** loads security context.
- **UsernamePasswordAuthenticationFilter —** authenticates user credentials.
- **AuthorizationFilter —** checks access permissions.
- **ExceptionTranslationFilter —** handles exceptions and errors.
- **FilterSecurityInterceptor —** final authorization decision.

WHAT HAPPENS AT EACH STEP

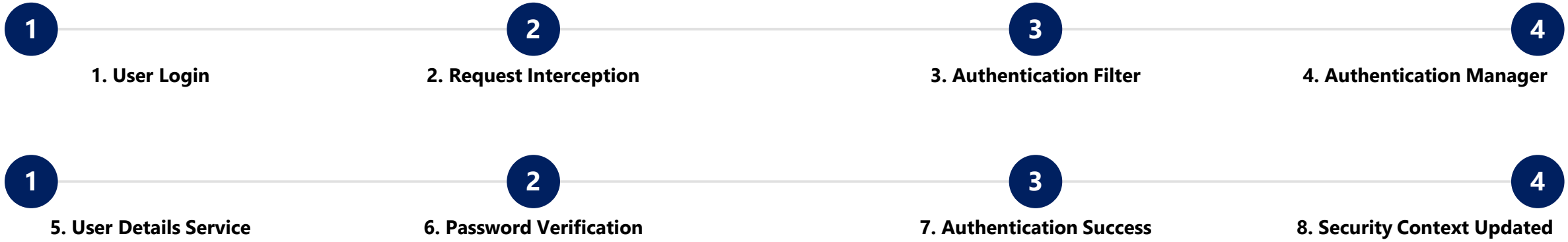
- 1. Request enters the Security Filter Chain.
- 2. User identity is verified (username/password or token validation).
- 3. User roles/permissions are checked against the requested resource.
- 4. Additional security checks are performed (CSRF, session, headers, etc.).
- 5. If everything is valid, the request reaches your application; the response is sent back to the client.

KEY TAKEAWAY The request processing flow ensures that every request is authenticated, authorized, and validated before reaching your application -- keeping your data and users secure.

USER AUTHENTICATION WORKFLOW (1 OF 2)

Spring Security provides a secure and flexible authentication process to verify the identity of users before granting access.

Authentication answers the question "Who are you?" Only authenticated users can access protected resources.



KEY COMPONENTS

- **AuthenticationFilter —** processes login requests and creates authentication tokens.
- **AuthenticationManager —** coordinates authentication providers to validate the token.
- **UserDetailsService —** loads user-specific data (username, password hash, roles).
- **PasswordEncoder —** encodes and verifies passwords securely.

KEY TAKEAWAY On success, an authenticated Authentication object is created and stored in the SecurityContext -- the user is logged in and can access protected resources.

USER AUTHENTICATION WORKFLOW (2 OF 2)

Authentication outcomes, and the security best practices that keep the workflow safe.

SEQUENCE (WHO TALKS TO WHOM)

#	Step
1	User submits login -> Filter Chain.
2-3	Filter Chain -> AuthenticationFilter -> AuthenticationManager: authenticate.
4-6	AuthenticationManager -> UserDetailsService -> Database: load user.
7	AuthenticationManager validates the password.
8-9	Success: SecurityContext is set; response returned to user.

AUTHENTICATION OUTCOMES

- **Success —** SecurityContext updated, user authenticated, access granted to protected resources.
- **Failure —** invalid credentials, authentication fails, 401 Unauthorized, user remains unauthenticated.

BEST PRACTICE

Strong Password Policies | **Use PasswordEncoder (BCrypt)** | **Never Store Plain Text** | **Account Lockout on Failures** | **Enable HTTPS**

KEY TAKEAWAY Best practice: always use strong password policies, PasswordEncoder (e.g. BCrypt), never store plain text passwords, implement account lockout on failures, and enable HTTPS.

LOGIN PROCESS

Spring Security handles the login process to authenticate users and establish a secure session (or issue a token) for accessing the application.

Purpose: verify user credentials and create an authenticated session (or JWT) so the user can access protected resources.

1. **Enter Credentials**
2. **Request Interception**
3. **Authentication Filter**
1. **Authentication Manager**
2. **Credential Verification**
3. **Login Success**

OUTCOMES

- **Success —** user is authenticated; SecurityContext (or JWT) is created; user is granted access.
- **Failure —** invalid username or password; authentication fails; error is shown; user remains unauthenticated.

BEST PRACTICES

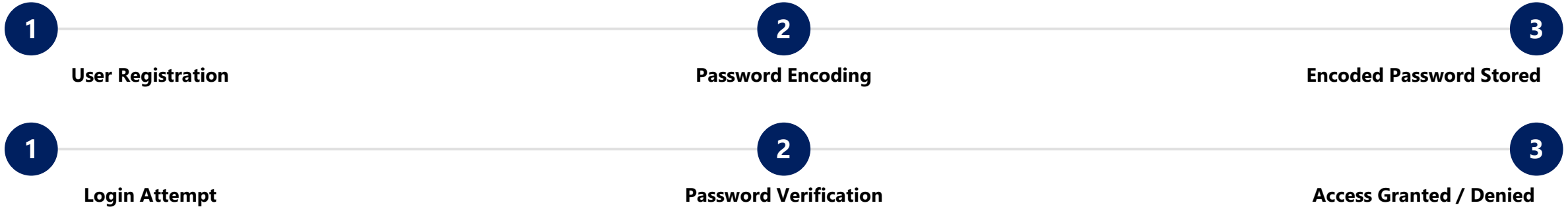
- Use strong password policies; use PasswordEncoder (e.g. BCrypt).
- Do not reveal whether the username exists (generic error messages).
- Implement account lockout on repeated failures.
- Always use HTTPS for login.

KEY TAKEAWAY Verify credentials, encode passwords, and never leak which field was wrong -- a secure login process is the front door to your entire application.

PASSWORD ENCODING (1 OF 2)

Spring Security never stores plain text passwords. It encodes passwords using a secure hashing algorithm with a salt to protect user credentials.

Purpose: protect user passwords even if the database is compromised; store passwords in a one-way, irreversible format; salt and hash make each password unique and secure.



HOW PASSWORD ENCODING WORKS

- Input password -> a random salt is generated for each password.
- Hashing algorithm (BCrypt, Argon2, PBKDF2, ...) combines password + salt.
- Same password + different salt = different encoded password.

KEY TAKEAWAY Password encoding is the first line of defense for protecting user credentials. Always encode, never store plain text.

PASSWORD ENCODING (2 OF 2)

Choosing a PasswordEncoder, best practices, and a working BCryptPasswordEncoder example.

PASSWORD ENCODERS IN SPRING SECURITY

Encoder	Strength	Use Case
BCryptPasswordEncoder	Strong	Recommended (default choice).
Pbkdf2PasswordEncoder	Strong	Good for compliance requirements.
Argon2PasswordEncoder	Very Strong	Best for new applications.
NoOpPasswordEncoder	Very Weak	Testing only -- never production.

SecurityConfig.java

```
@Bean
public PasswordEncoder passwordEncoder() {
  return new BCryptPasswordEncoder(10);
  // 10 = strength (log rounds)
}
```

BEST PRACTICES

- Always use a strong PasswordEncoder like BCrypt (the default choice).
- Never store or log plain text passwords, in code, notes, or screenshots.
- Use the same PasswordEncoder for encoding and matching.
- Ensure an adequate work factor (strength) for better security; a unique salt per password is handled automatically by Spring Security.

KEY TAKEAWAY Never use MD5, SHA-1, or NoOpPasswordEncoder for passwords in production -- always use a modern, adaptive hashing algorithm like BCrypt or Argon2.

ROLE-BASED ACCESS CONTROL (RBAC) (1 OF 2)

RBAC restricts system access based on the roles assigned to users. Users inherit permissions through roles, simplifying access management and enforcing least privilege.

RBAC is an authorization model that grants permissions to users based on their roles. Roles are assigned permissions, and users are assigned roles.

BENEFITS OF RBAC

Improved Security

Users have only the access they need.

Simplified Management

Manage access by roles, not individual permissions.

Scalability

Easily add new users and roles as the system grows.

Auditability

Clear visibility of who has access to what and why.

KEY TAKEAWAY RBAC centralizes permission management through roles, improving security, simplifying administration, and ensuring users get the right access to the right resources.

ROLE-BASED ACCESS CONTROL (RBAC) (2 OF 2)

A worked example of user -> role -> permission mapping, how the access decision works, and best practices.

EXAMPLE: USER -> ROLE -> PERMISSION

User	Role	Permission
agent1	AGENT	Read/write CUS-1001, CUS-1002 via /api/customers/**
admin1	ADMIN	All AGENT permissions + /api/admin/** access

HOW THE ACCESS DECISION WORKS

- 1. User attempts to access a protected resource.
- 2. Is the user authenticated? No -> Access Denied (401).
- 3. Does the user's role satisfy the required permission? No -> Access Denied (403).
- 4. Yes to both -> Access Granted; resource is returned.

RBAC BEST PRACTICES

- **Design Roles Around Business Functions —** not technical permissions.
- **Follow the Principle of Least Privilege —** grant the minimum permissions required.
- **Use Role Hierarchies (if needed) —** avoid duplication of permissions.
- **Review Roles and Permissions Regularly —** remove unused roles and outdated permissions.

KEY TAKEAWAY RBAC centralizes permission management through roles, improving security, simplifying administration, and ensuring users get the right access to the right resources.

USER ROLES AND PERMISSIONS

Roles group users with similar responsibilities. Permissions define what actions can be performed on specific resources.

COMMON USER ROLES

Role	Description
ADMIN	Full system access; can manage users, roles, and all resources.
MANAGER	Can manage data and processes within their domain.
USER / AGENT	Can access and manage assigned resources.
AUDITOR	Read-only access for monitoring and reporting.
GUEST	Limited access to public or non-sensitive resources.

EXAMPLE PERMISSIONS

Permission	Allows
VIEW / READ	Viewing dashboards, reports, or records.
CREATE	Creating new data (e.g. a new customer).
UPDATE	Modifying existing data or settings.
DELETE	Removing records.
MANAGE	Managing users, roles, and permissions.

BEST PRACTICES

- **Define Roles by Business Function**
- **Grant Minimum Permissions (Least Privilege)**
- **Review Regularly**
- **Avoid Hardcoding Roles**
- **Use Meaningful Names**

KEY TAKEAWAY Proper use of roles and permissions ensures secure, scalable, and maintainable access control. Assign the right roles, grant the right permissions, and review regularly.

SECURING ENDPOINTS (1 OF 2)

Spring Security lets you protect application endpoints (URLs/APIs) so that only authenticated and authorized users can access them.

Purpose: protect sensitive endpoints from unauthorized access, enforce authentication and authorization rules, and apply the principle of least privilege.

SecurityConfig.java (matcher-based endpoint protection -- Lab 28 pattern)

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
  http.csrf(csrf -> csrf.disable())
      .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
          .requestMatchers("/api/auth/login", "/actuator/health").permitAll()
          .requestMatchers("/api/admin/**").hasRole("ADMIN")
          .requestMatchers("/api/customers/**").hasAnyRole("AGENT", "ADMIN")
          .anyRequest().authenticated())
      .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
  return http.build();
}
```

KEY TAKEAWAY Default deny: allow only what is necessary. Authenticate first, then authorize.

SECURING ENDPOINTS (2 OF 2)

Endpoint protection example, method-level security annotations, and techniques to secure endpoints.

EXAMPLE: ENDPOINT PROTECTION (LAB 28)

Endpoint	Method	Access Rule	Role
/api/auth/login	POST	permitAll()	None (public)
/api/customers/**	GET/POST	hasAnyRole(...)	AGENT or ADMIN
/api/admin/**	GET	hasRole('ADMIN')	ADMIN
anyRequest()	*	authenticated()	Any logged-in user

Method-Level Security (Alternative)

```
@RestController
@RequestMapping("/api/admin")
public class AdminController {
  @GetMapping("/customers")
  @PreAuthorize("hasRole('ADMIN')")
  public List<CustomerResponse> adminList() {
    // ...
  }
}
```

TECHNIQUES TO SECURE ENDPOINTS

- **Authentication**
- **Authorization**
- **Role-Based Access Control**
- **HTTP Method Restrictions**
- **Request Matchers**
- **Custom Access Rules (SpEL)**

KEY TAKEAWAY Securing endpoints ensures users can access only the resources they are allowed to. Authenticate first, then authorize.

WHAT IS JWT? (1 OF 2)

JWT (JSON Web Token) is a compact, URL-safe token format used to securely transmit information between parties as a JSON object.

Commonly used for authentication and authorization in stateless applications (e.g. Spring Security with REST APIs).

WHY USE JWT?

- **Stateless —** no server-side session storage required.
- **Scalable —** works across distributed systems and microservices.
- **Secure —** digitally signed to prevent tampering.
- **Compact —** small in size and easy to transmit (e.g. in HTTP headers).

HOW JWT WORKS

HTTP Header Example

```
Authorization: Bearer <JWT_TOKEN>
```

KEY TAKEAWAY JWT is a secure and efficient way to transmit authentication and authorization information between client and server -- stateless, scalable, and widely used in modern REST APIs.

WHAT IS JWT? (2 OF 2)

Common claims, where JWT is used, and its advantages and limitations.

COMMON JWT CLAIMS (PAYLOAD)

Claim	Meaning
sub	Subject (user ID / username, e.g. “agent1”).
role	User role(s), e.g. AGENT or ADMIN.
iat	Issued at (timestamp).
exp	Expiration time.
iss / aud	Issuer / audience.

WHERE JWT IS USED

- Authentication for REST APIs
- Microservices communication
- Single Sign-On (SSO)
- Mobile and SPA applications

ADVANTAGES

- Stateless -- no server-side session storage.
- Self-contained and cross-language compatible.
- Digitally signed -- cannot be tampered with silently.

LIMITATIONS

- Cannot be revoked easily (until expiration).
- Sensitive data should never be stored in the payload.
- Must use HTTPS to prevent token interception.

KEY TAKEAWAY JWT enables stateless security and scalability -- but never store secrets in the payload, and always use HTTPS.

JWT STRUCTURE

A JSON Web Token consists of three parts separated by dots (.), each Base64Url encoded.

1. HEADER

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. PAYLOAD (CLAIMS)

```
{
  "sub": "agent1",
  "role": "AGENT",
  "iat": 1717152345,
  "exp": 1717155945
}
```

3. SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secretKey
)
```

ALGORITHM TYPES

Type	Description	Example
HMAC (Symmetric)	Uses a shared secret key -- both issuer and verifier hold the same key.	HS256 (Lab 28's choice)
RSA / ECDSA (Asymmetric)	Uses a private/public key pair -- issuer signs with private, others verify with public.	RS256, ES256

KEY TAKEAWAY If any part of the token is changed, signature verification fails and the token is rejected -- keep the secret key private and secure.

JWT AUTHENTICATION WORKFLOW (1 OF 2)

JWT enables stateless authentication. Once the user is authenticated, the server issues a token which is used for all subsequent requests.

CLIENT

- 1. Login Request -- user submits credentials.
- 5. Store Token -- client stores the JWT securely.

SERVER

2. Validate	3. Generate	4. Return	6-9. Verify
Server validates username and password.	If valid, generates a signed JWT.	Returns the JWT to the client.	Validates JWT + role on each request.

Authenticated Request

```
GET /api/customers/CUS-1001
Authorization: Bearer <JWT_TOKEN>
X-Correlation-Id: lab-request-001
```

KEY TAKEAWAY JWT allows secure, stateless authentication by issuing a signed token that the client uses for all subsequent requests until it expires.

TRY IT YOURSELF -- EXERCISE 3: JWT LOGIN TODOs (STARTER)

Reinforces: completing a JWT login/filter sketch -- a fill-in-the-blank TODO starter, not a from-scratch build.

STEP	WHAT TO DO
Starter	Create notes/JwtSecuritySketch.java -- a TODO / fill-in-the-blank skeleton is provided; blanks do not compile as-is.
Fill: login()	Replace _____ so jwtService.issue/createToken(username) returns a token, and the response Map returns it as "accessToken".
Fill: filter	Replace _____ so the filter checks authorizationHeader.startsWith("Bearer ") before parsing the token.
Secret hygiene	In .env.example-style notes, write JWT_SECRET=_____ as a placeholder only -- never a real production secret.
Reflect	SOAP UsernameToken (Lab 24) is a separate mechanism from REST JWT -- do not conflate the two.
Reference	exercises/exercise-03-jwt-login-todos.md

JwtSecuritySketch.java (fill every _____) -- STARTER, not a solution

```
class AuthController {
  String login(String u, String p) {
    String token = jwtService._____(u); // issue/create
    return Map.of("accessToken", _____);
  }
}
// Filter: require header.startsWith(_____) == "Bearer "
// then token = authHeader.substring(7); parse + set context
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 3 (STARTER) -- fill in every _____ and // TODO, then re-read your sketch -- see exercises/exercise-03-jwt-login-todos.md.

JWT AUTHENTICATION WORKFLOW (2 OF 2)

Validating and authorizing an incoming request, the token lifecycle, and important notes.

VALIDATE -> AUTHORIZE -> RESPOND

1. **Validate JWT**
   Signature, expiry
2. **Authorize Request**
   Role check
3. **Return Response**
   Protected resource or 401/403

TOKEN LIFECYCLE

1. **Login**
2. **Token Issued**
3. **Stored on Client**
4. **Sent in Requests**
5. **Validated by Server**

IMPORTANT NOTES

- **Keep Secret Private**
- **Short Expiration + Refresh**
- **Always Use HTTPS**
- **No Sensitive Data in Token**

KEY TAKEAWAY JWT allows secure, stateless authentication by issuing a signed token that the client uses for all subsequent requests until it expires.

TOKEN VALIDATION (JWT)

Token validation is the process of verifying a JWT to ensure it is authentic, not expired, and safe to use.

Goal: ensure the token is issued by a trusted server, is valid, not expired, and contains the expected claims.

TOKEN VALIDATION STEPS

- **1. Extract Token —** extract the JWT from the Authorization header (Bearer token).
- **2. Parse Token —** decode into Header, Payload, and Signature.
- **3. Verify Signature —** verify using the server's secret key.
- **4. Validate Claims —** check standard claims like exp, iat, sub.
- **5. Custom Validation —** validate roles/permissions as per business rules.
- **6. Result —** if all validations pass, access is granted; otherwise, reject the request.

IF VALIDATION FAILS

Failure	Result
Invalid Signature	Token is tampered or not issued by trusted server -- 401.
Expired Token	Token has expired -- user must log in again -- 401.
Invalid Claims	One or more claims invalid (wrong issuer/audience) -- 401.
Insufficient Role	Token is valid but role does not satisfy the rule -- 403.

KEY TAKEAWAY Token validation ensures that only genuine and valid tokens are accepted. Always validate signature, expiration, and important claims before trusting the token.

TRY IT YOURSELF -- EXERCISE 4: MOCKMVC EVIDENCE MATRIX

Reinforces: drafting the 401/403/200 status matrix Lab 28's automated MockMvc tests must prove -- design only, no test code yet.

STEP	WHAT TO DO
Goal	Draft the status matrix Lab 28's MockMvc tests must cover, in notes/mockmvc-matrix.md.
Scenarios	Five rows: anonymous GET customers; bad token; agent GET customer; agent GET admin; admin GET admin.
Expected statuses	Fill the expected 401/403/200 result for each of the five scenarios.
Fixture	The successful customer-read scenario uses CUS-1001 (Amina Khan).
Boundary	Do not write full MockMvc test code in this pre-lab exercise -- plan the matrix only.
Reference	exercises/exercise-04-mockmvc-matrix.md

Status Matrix Skeleton (notes/mockmvc-matrix.md)

```
Scenario                        Expected
anonymous GET /api/customers/** 401
invalid/bad Bearer token        401
agent1 GET CUS-1001             200
agent1 GET /api/admin/**        403
admin1 GET /api/admin/**        200
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 4 before continuing -- fill in all five expected statuses using CUS-1001 as the fixture -- see exercises/exercise-04-mockmvc-matrix.md.

SECURITY BEST PRACTICES (1 OF 2)

Following security best practices helps protect your application, users, and data from common threats and vulnerabilities.

Category	Practices
1. Authentication	Use strong password policies; enforce MFA; avoid user enumeration; use secure login endpoints (HTTPS only).
2. Authorization	Implement RBAC; follow least privilege; validate permissions on the server for every request; never rely on client-side checks.
3. Protect Data in Transit	Enforce HTTPS for all endpoints; use TLS 1.2+; redirect HTTP to HTTPS; use HSTS.
4. Protect Data at Rest	Encrypt sensitive data in the database; store passwords with strong hashing (BCrypt, Argon2); manage encryption keys securely; never store secrets in code or configuration.
5. JWT and Token Best Practices	Use short-lived access tokens; store tokens securely (HttpOnly, Secure cookies preferred); validate signature, expiration, and claims; implement refresh token rotation.
6. Input Validation and Sanitization	Validate and sanitize all inputs; use server-side validation; prevent injection attacks; use parameterized queries and prepared statements.

KEY TAKEAWAY Security is not a one-time task. It is an ongoing process that involves people, processes, and technology. Secure by design. Verify always. Trust never.

SECURITY BEST PRACTICES (2 OF 2)

Practices 7-12: session management through secure development.

Category	Practices
7. Session Management	Use secure, HttpOnly, SameSite cookies where sessions are used; set appropriate timeouts; invalidate on logout; regenerate session ID after login.
8. Error Handling	Do not expose stack traces or internal details; return meaningful but safe error messages; log errors securely for monitoring.
9. Dependency and Patch Management	Keep all dependencies and libraries up to date; regularly scan for known vulnerabilities; apply security patches promptly.
10. Rate Limiting and Brute-Force Protection	Limit login attempts and API requests; use lockout policies for repeated failures; monitor and alert on anomalies.
11. Logging and Monitoring	Log security events (login, access denied, token failures); avoid logging sensitive data (passwords, tokens); use centralized logging tools.
12. Secure Development Practices	Follow secure coding standards; conduct threat modeling; perform regular security testing (SAST, DAST, penetration testing).

KEY TAKEAWAY Security is not a one-time task. It is an ongoing process that involves people, processes, and technology. Secure by design. Verify always. Trust never.

COMMON SECURITY MISTAKES (1 OF 2)

Avoid these common mistakes to build secure and resilient applications.

Mistake	Avoid It By
1. Weak or Default Passwords	Enforce strong password policies; use MFA for better protection.
2. Missing Authorization	Only authenticating users but not checking permissions -- always enforce authorization on every endpoint (least privilege).
3. Exposing Sensitive Information	Returning stack traces, SQL errors, or internal details -- log securely on the server side instead.
4. Insecure Data Transmission	Using HTTP instead of HTTPS -- always use HTTPS and HSTS; redirect all HTTP traffic to HTTPS.
5. Improper Token Management	Storing tokens in localStorage or exposing them in URLs -- prefer HttpOnly/Secure cookies; set proper expiration and refresh rotation.
6. No Input Validation or Sanitization	Trusting user input leads to SQL Injection, XSS, Command Injection -- validate, sanitize, and use parameterized queries.

KEY TAKEAWAY Most security breaches happen due to simple mistakes that could have been avoided. Follow best practices, review your code, and stay vigilant.

COMMON SECURITY MISTAKES (2 OF 2)

Mistakes 7-12: hardcoded secrets through outdated dependencies.

Mistake	Avoid It By
7. Hardcoding Secrets in Code	Storing passwords, API keys, secrets in code or config files -- use environment variables, Vault, or a secret manager.
8. Overly Permissive CORS Settings	Allowing all origins, methods, and headers -- restrict CORS to trusted origins only; avoid wildcards in production.
9. Not Protecting Against CSRF	Missing CSRF protection on state-changing requests -- use CSRF tokens or SameSite cookies where sessions are used; for stateless Bearer APIs, document why CSRF is disabled.
10. Insufficient Session Management	Long session timeouts and not invalidating sessions -- set proper timeouts; invalidate on logout and password change.
11. Excessive Privileges to Application	Running apps with admin or root privileges -- use dedicated service accounts with minimal permissions.
12. Outdated Dependencies and Libraries	Using libraries with known vulnerabilities -- regularly update dependencies; scan for vulnerabilities continuously.

KEY TAKEAWAY Security is not a feature, it is a mindset. Design securely, validate always, and monitor continuously.

PRODUCTION SECURITY CONSIDERATIONS (1 OF 2)

Security in production requires more than secure code. It involves people, processes, infrastructure, and continuous monitoring.

Consideration	What It Involves
1. Secure Configuration	Harden server/OS configuration; disable unnecessary services and default accounts; apply least privilege; use security headers (HSTS, CSP).
2. Strong Authentication and Authorization	Enforce MFA for users and admins; use strong password policies; implement RBAC and least privilege; review roles and permissions regularly.
3. Secure Communication	Enforce HTTPS (TLS 1.2+); redirect all HTTP traffic to HTTPS; use HSTS to prevent protocol downgrade; validate SSL/TLS certificates.
4. Protect Secrets and Sensitive Data	Store secrets in a dedicated secrets manager (Vault, AWS Secrets Manager); never store secrets in code, images, or environment files; rotate keys and secrets regularly.
5. Input Validation and Output Encoding	Validate and sanitize all user input; prevent SQL Injection, XSS, CSRF, Command Injection; use parameterized queries; encode output to prevent XSS.

KEY TAKEAWAY A secure application builds trust, protects data, and ensures business continuity -- security is an ongoing process, not a one-time task.

PRODUCTION SECURITY CONSIDERATIONS (2 OF 2)

Considerations 6-10, plus additional production safeguards.

Consideration	What It Involves
6. Secure Token and Session Management	Use short-lived access tokens and refresh tokens; store tokens in HttpOnly, Secure, SameSite cookies where applicable; invalidate tokens on logout/password change; implement token revocation.
7. Logging, Monitoring, and Alerting	Log security events (login, access denied, config changes); avoid logging sensitive data; monitor logs in real time; alert on suspicious activity and anomalies.
8. Patch Management and Vulnerability Management	Keep OS, dependencies, and frameworks up to date; scan for known vulnerabilities regularly; apply security patches promptly.
9. Backups and Disaster Recovery	Maintain encrypted, tested backups in a separate secure location; test restore procedures regularly; have a documented incident response and DR plan.
10. Access Control and Auditing	Limit access to production systems and data; use bastion hosts/VPN for access; enable audit trails for critical actions; review access regularly.

ADDITIONAL CONSIDERATIONS

Web Application Firewall (WAF) **API Rate Limiting** **Network Security** **Regular Security Testing (SAST/DAST/Pen Test)**

KEY TAKEAWAY Defense in depth protects your systems from multiple layers. Monitor continuously, adapt, and improve.

TRY IT YOURSELF -- EXERCISE 5: PRODUCTION IDP CHECKLIST

Reinforces: drafting the production IdP and key-rotation checklist that separates Lab 28's teaching setup from a real deployment.

STEP	WHAT TO DO
Goal	Draft docs/security-notes.md outline items for a production IdP and key rotation.
Outline	In notes/security-notes-outline.md: replace lab users with a real IdP; rotate signing keys; use short token TTL; enforce HTTPS only.
Lab vs. prod	State explicitly that in-memory agent1/admin1 users are lab-only, never production accounts.
Transfers	Note that Lab 27's money-transfer routes must stay behind auth in production narratives too.
Boundary	Do not implement a full OAuth2 Authorization Server here -- outline only.
Reference	exercises/exercise-05-production-checklist.md

security-notes-outline.md (four checklist items)

```
1. Replace in-memory agent1/admin1 with a real IdP
2. Rotate JWT signing keys on a schedule
3. Use a short access-token TTL (+ refresh flow)
4. Enforce HTTPS everywhere -- no plain HTTP
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 5 before continuing -- draft all four checklist items and mark agent1/admin1 as lab-only -- see exercises/exercise-05-production-checklist.md.

TRY IT YOURSELF -- EXERCISE 6: LAB 28 READINESS CHECKLIST

Capstone: confirm your Labs 25-27 CRM baseline is ready before Lab 28 layers Spring Security on top.

STEP	WHAT TO DO
Goal	Ready lab28-crm by confirming the Labs 25-27 concepts it depends on, in notes/lab28-readiness.md.
Depends on	A runnable Customer REST API from Labs 25-27 -- confirm it starts and responds before adding security.
Path	Write the working folder path: examples/lab28-crm.
Evidence plan	Plan where login + Bearer GET CUS-1001 screenshots will be saved as evidence.
Defer	Global Bean Validation / ErrorResponse unification waits for Lab 29 -- do not build it now.
Reference	exercises/exercise-06-lab28-readiness.md

Readiness Checklist (notes/lab28-readiness.md)

```
[ ] Customer REST API from Labs 25-27 runs cleanly
[ ] examples/lab28-crm path created
[ ] Evidence folder planned for login + Bearer GET screenshots
[ ] ErrorResponse unification deferred to Lab 29
```

KEY TAKEAWAY TRY IT NOW Complete Exercise 6 before Lab 28 -- confirm your CRM baseline is ready and note what's deferred to Lab 29 -- see exercises/exercise-06-lab28-readiness.md.

LAB OVERVIEW -- SPRING SECURITY FUNDAMENTALS

Lab 28: Spring Security Basics -- Northstar CRM JWT and Roles

BUSINESS SCENARIO

- Leadership will not expose customer APIs on the open network -- unauthenticated callers must be rejected.
- Agents (AGENT) work Amina/Ravi records within policy; admins (ADMIN) get elevated operations.
- Authn/authz must be automated so new routes do not silently ship open.

LEARNING OBJECTIVES

- Add Spring Security to a Spring Boot 3 CRM API; issue a JWT on login.
- Validate JWTs on every request with a filter; deny by default.
- Protect /api/customers/** for AGENT/ADMIN and /api/admin/** for ADMIN only.
- Prove the 401/403/200 matrix with MockMvc, keeping secrets out of Git.
- Explain the trust boundary between login, token issuance, and authorization.

WHAT YOU BUILD

- Copy prior CRM to lab28-crm; add Security + JWT deps; build a stateless SecurityFilterChain, JwtService, JwtAuthenticationFilter, and /api/auth/login; enforce AGENT/ADMIN matchers; write MockMvc 401/403/200 tests; document production IdP/secret-rotation notes.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; lab users agent1 (AGENT) and admin1 (ADMIN) prove role separation.

LAB TASKS AND DELIVERABLES

Northstar CRM JWT and Roles -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch prior CRM to lab28-crm; pin Security + JWT dependencies; no secrets committed.
2	Configure the stateless SecurityFilterChain -- deny by default.
3	Implement UserDetails and BCrypt password encoding (agent1, admin1).
4	Implement JwtService -- issue and parse tokens (HS256, sub/roles/iat/exp).
5	Build AuthController.login() -- verify credentials, return a JWT.
6	JWT filter validates Bearer tokens; authenticated customer access works.
7	Role separation -- AGENT vs ADMIN, proving 403 vs 200.
8-10	Automated MockMvc 401/403/200 matrix; runbook + production IdP notes; failure experiments + evidence pack.

EXPECTED DELIVERABLES

- lab28-crm with SecurityFilterChain, JWT login, AGENT/ADMIN roles.
- MockMvc evidence for 401/403/200.
- Successful-path evidence (login + CUS-1001 with AGENT).
- Controlled-failure evidence (401/403).
- Auth-flow notes (docs/security-notes.md) + production IdP/secret-rotation checklist; no secrets or target/ committed.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Production 10 · Docs 10

KEY TAKEAWAY Shipping an open /api/customers/** is an honor violation. Logging raw JWTs or committing secrets loses security marks. Cannot distinguish 401 from 403 -> incomplete.

MODULE 28 SUMMARY

In this module, we explored how Spring Security provides authentication, authorization, and protection for enterprise Java applications.

KEY CONCEPTS COVERED

Security Basics

Spring Security's comprehensive framework for authn, authz, and protection.

Authentication

Verifying identity via credentials, password encoding, and JWT.

Authorization

Role-based access control -- only authorized users reach protected resources.

Secured Endpoints

Public endpoints stay open; protected endpoints require authentication and roles.

Configuration

SecurityFilterChain, in-memory/JWT users, and password encoding.

Best Practices

Least privilege, strong passwords, secure tokens, and validated input.

MODULE FLOW RECAP

1. **Authn (who)**
2. **Authz (what)**
3. **Filter Chain**
4. **JWT**
5. **Lab: Northstar CRM**

KEY TAKEAWAY You can now build secure, role-based Spring Boot applications that protect resources, authenticate users, and enforce proper access control.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of Spring Security's purpose, authentication, and configuration.

1. What is the primary purpose of Spring Security?

A. To connect to a database

B. To add security to Spring applications

C. To build REST APIs

D. To manage application logs

2. Which component is responsible for authenticating users?

A. AuthorizationManager

B. AuthenticationManager

C. PasswordEncoder

D. SecurityContextHolder

3. What type of authentication does Lab 28's Northstar CRM implement?

A. HTTP Basic Authentication

B. Form-based session authentication

C. OAuth2 Authentication

D. JWT (stateless Bearer token) Authentication

4. Where do we configure security settings in a Spring Boot application?

A. application.properties

B. web.xml

C. SecurityConfig.java (a SecurityFilterChain @Bean)

D. logback.xml

KEY TAKEAWAY Spring Security exists to add authentication and authorization to Spring applications -- the AuthenticationManager verifies identity, and SecurityConfig wires it all together.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on roles, password encoding, unauthenticated access, and logout.

5. Which annotation is commonly used to secure endpoints based on roles?

A. @Secured

B. @RolesAllowed

C. @PreAuthorize

D. @EnableSecurity

6. What does an AGENT role allow access to in Lab 28's Northstar CRM?

A. Only /api/admin/** endpoints

B. Public endpoints and /api/customers/**

C. Only database operations

D. All endpoints without a token

7. Which component is used to encode passwords in Spring Security?

A. UserDetails

B. PasswordEncoder

C. AuthenticationProvider

D. UserDetailsService

8. What happens when a user calls a protected endpoint with no JWT at all?

A. Access is granted

B. 401 Unauthorized is returned

C. 403 Forbidden is returned

D. 500 Internal Server Error is returned

KEY TAKEAWAY PasswordEncoder secures credentials; @PreAuthorize enforces roles; a missing token means 401, not 403.

KNOWLEDGE CHECK (3 OF 3)

Two final questions, plus what these ten questions prove together.

9. Which endpoint is typically used to handle a JWT logout / client-side token discard?

A. /logout

B. /exit

C. /signout

D. /logoutUser

10. Which interface is used to load user-specific data for authentication?

A. UserDetailsService

B. AuthenticationService

C. UserService

D. SecurityService

WHAT THESE TEN QUESTIONS PROVE TOGETHER

- **Vocabulary —** authentication versus authorization, and the components that implement each (AuthenticationManager, PasswordEncoder, UserDetailsService).
- **Configuration —** security is wired declaratively in a SecurityFilterChain bean, not scattered through Controller code.
- **Failure Modes —** a missing token is 401; an insufficient role is 403 -- and the difference is instructor-verified in Lab 28.
- **Real-World Grounding —** the AGENT/ADMIN roles and JWT design in this quiz are exactly what Lab 28 requires you to build.

KEY TAKEAWAY Score 7 or more out of 10 to demonstrate a good understanding of Spring Security basics before starting Lab 28.

"Authenticate first, then authorize -- every request, every time."

MODULE 28 COMPLETE

Spring Security Fundamentals

You can now build secure, role-based Spring Boot applications -- authenticating users, encoding passwords, issuing and validating JWTs, and enforcing access control at every endpoint.